

Tutorial - Semana 2, Encontro 2

Introdução

No Encontro 1, o Player (CharacterBody3D) foi montado dentro de `level_exploration.tscn` — sólido, alinhado ao chão, e já chamando `move_and_slide` a cada frame de física, mas ainda sem receber nenhuma velocidade. Este encontro resolve o problema restante: como traduzir uma tecla pressionada pelo jogador em movimento do personagem, sem que o código de gameplay precise saber qual tecla física foi usada? O Godot resolve isso com o **Input Map**, uma camada de abstração entre o dispositivo físico e a ação lógica do jogo.

Este tutorial dá continuidade direta ao Encontro 1 — o Player já deve existir na Scene, parado, antes de começar.

Objetivos da semana

- Explicar Input Map e InputEvent como camada de desacoplamento entre dispositivo físico e ação lógica.
- Configurar um Input Map para movimentação.
- Conectar o Input Map ao `move_and_slide` do Player.

Resultado esperado ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), cada estudante terá um Player (CharacterBody3D) movendo-se no nível de teste através de um Input Map próprio, com pelo menos uma Action adicional não demonstrada em aula. Este tutorial cobre apenas o **Encontro 2**: a configuração do Input Map e a conexão com a movimentação.

Pré-requisitos

- Player (CharacterBody3D) montado na Scene `level_exploration.tscn`, com `CollisionShape3D`, `Malha` e Orchestration `player.torch` chamando `move_and_slide` (ver Tutorial - Semana 2, Encontro 1).
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com o Player parado, porém fisicamente funcional, dentro de `level_exploration.tscn`.

Arquivos necessários

- Nenhum arquivo externo. O Input Map é configurado inteiramente dentro do Project Settings do Godot.

Assets utilizados

- Nenhum. Os pacotes Kenney começam a ser utilizados a partir da Semana 3.

Projeto esperado

- Projeto aberto no Godot 4.7, com o Player do Encontro 1 pronto para receber input.
- Orchestration `player.torch` já existente, com o evento `PhysicsProcess` chamando `move_and_slide`.

Imagem sugerida

Objetivo: mostrar a aba Input Map do Project Settings, com as Actions de movimentação já configuradas. Enquadramento: captura de tela da janela Project Settings, aba Input Map.

Elementos importantes: lista de Actions (`move_forward` , `move_back` , `move_left` , `move_right`), teclas associadas a cada uma. Destaque: o botão "Add" usado para criar uma nova Action. Legenda sugerida: "Input Map do projeto com as Actions de movimentação configuradas."

Parte 1 — Input Map e InputEvent como camada de desacoplamento

Objetivo

Entender por que engines modernas inserem uma camada de abstração entre a tecla física pressionada e a ação lógica do jogo, antes de configurar qualquer Action no editor.

Conceito

Se o código de gameplay lesse diretamente "tecla W pressionada", qualquer remapeamento de controles ou suporte a um gamepad exigiria reescrever a lógica do jogo inteira, tecla por tecla. Por isso, engines modernas inserem uma camada de abstração entre o dispositivo físico e a ação lógica: o jogador aperta uma tecla, a engine traduz isso em uma **Action** nomeada (por exemplo, "mover para frente"), e é essa Action — não a tecla em si — que o código de gameplay consome.

No Godot, essa camada é o **Input Map**, configurado uma única vez no projeto (em **Project Settings > Input Map**), associando uma ou mais teclas/botões físicos a cada Action. Em tempo de execução, cada tecla pressionada gera um **InputEvent**, que o Godot já traduz automaticamente para a Action correspondente — o script ou a Orchestration não precisam tratar o evento bruto, apenas perguntar "a Action `move_forward` está pressionada agora?".

Passo a passo

1. Sem abrir o Project Settings ainda, discuta com a turma: se o código do jogo checasse diretamente a tecla `W`, o que aconteceria ao tentar oferecer suporte a um gamepad?
2. Abra **Project > Project Settings** e selecione a aba **Input Map**.
3. No campo de texto no topo, digite `move_forward` e clique em **Add**.
4. Com a Action `move_forward` criada, clique no ícone de `+` ao lado dela para adicionar um evento de tecla, pressione a tecla **W** e confirme.
5. Repita o processo para criar `move_back` (tecla **S**), `move_left` (tecla **A**) e `move_right` (tecla **D**).
6. Feche o Project Settings e reabra a Orchestration `player.torch` criada no Encontro 1.
7. Dentro do PhysicsProcess já existente, adicione nós do Orchestrator que leem o estado das quatro Actions (equivalente a `Input.get_action_strength()` em GDScript).
8. Combine os valores lidos das quatro Actions em um vetor de direção 2D (frente/trás e esquerda/direita).

9. Transforme esse vetor de direção em uma velocidade 3D, multiplicando por uma velocidade constante (por exemplo, 5.0) e atribuindo o resultado à propriedade `velocity` do `Player`.
10. Confirme que a chamada a `move_and_slide`, já existente desde o Encontro 1, permanece após a atribuição de `velocity`.
11. Salve a Orchestration e a Scene, e pressione **Play Scene** (F6).
12. Utilize as teclas W, A, S e D para mover o Player pelo nível de teste, confirmando que ele desliza corretamente ao encostar em obstáculos.

Resultado esperado

O Player se move pelo nível de teste em resposta às teclas W, A, S e D, através de quatro Actions (`move_forward`, `move_back`, `move_left`, `move_right`) configuradas no Input Map do projeto e lidas pela Orchestration `player.torch`, que aplica a direção resultante à `velocity` do `CharacterBody3D` antes de chamar `move_and_slide`.

Verificando

1. Abra **Project Settings > Input Map** e confirme que as quatro Actions existem, cada uma com a tecla correta associada.
2. Rode a Scene com F6 e mova o Player nas quatro direções, confirmando resposta imediata a cada tecla.
3. Encoste o Player em uma borda do `Chao` ou em outro obstáculo da cena e confirme que ele desliza ao longo da superfície, sem travar ou atravessar.

Problemas comuns

- Actions com nomes ambíguos ou duplicados no Input Map, causando comportamento inesperado: reforçar a convenção de nomes clara usada aqui (`move_forward`, `move_back`, `move_left`, `move_right`).
- Direção de movimento invertida (por exemplo, W move o Player para trás): checar a orientação do Node `Player` no Viewport antes de depurar a lógica de input — o eixo "frente" do `CharacterBody3D` depende de como ele foi rotacionado.
- Player não se move mesmo com o Input Map configurado: confirmar que a `velocity` está sendo atribuída antes da chamada a `move_and_slide`, e não depois — a ordem dentro do `PhysicsProcess` importa.
- Movimento "picotado" ou não suave: confirmar que a leitura de input e a chamada a `move_and_slide` estão de fato dentro do evento `PhysicsProcess`, e não em um evento de `Process` comum (por frame de renderização).

Boas práticas

- Nomear Actions por intenção do jogador (`move_forward`), nunca pela tecla física (`tecla_w`) — o nome da Action deve continuar fazendo sentido mesmo se o jogador remapear os controles.
- Centralizar toda leitura de input relacionada à movimentação dentro da Orchestration do próprio Player, evitando espalhar chamadas de Input Map por múltiplos Nodes.
- Testar cada Action isoladamente durante a configuração (uma tecla de cada vez) antes de combinar as quatro em um vetor de direção — isso facilita identificar qual Action está mal configurada em caso de erro.

Comparação com Unity

A Unity resolve o mesmo problema com o **Input System** (novo), usando **Action Maps** e um componente **Player Input** para conectar as Actions ao código — uma solução com mais camadas de configuração (Action Maps, Control Schemes, bindings por dispositivo) do que o Input Map do Godot. O Godot concentra tudo em um único Input Map global do projeto, mais simples de configurar para um caso como este, porém com menos granularidade nativa para múltiplos esquemas de controle simultâneos (por exemplo, dois jogadores locais com Action Maps distintos).

Ao final da semana

Ao final da Semana 2 (Encontros 1 e 2), o projeto do Vertical Slice deve conter:

- O Player (CharacterBody3D) montado no Encontro 1, com `CollisionShape3D` e `Malha` alinhados.
- Um Input Map do projeto com as Actions `move_forward` , `move_back` , `move_left` , `move_right` , mais uma Action adicional criada no desafio deste encontro.
- A Orchestration `player.torch` lendo o Input Map e aplicando a direção resultante a `move_and_slide` , tornando o Player efetivamente controlável.

Segundo o PROJECT_ARCHITECTURE.md (seção 6, Módulo 1), este resultado corresponde à conclusão dos itens "Player (locomoção)" e "Input do jogador", pré-requisitos diretos para a Cena de teste (graybox) e para a Renderização/build, que serão construídas na Semana 3, encerrando o Módulo 1.

Desafio

Adicione uma nova Action ao Input Map, não demonstrada neste tutorial — correr, agachar ou pular —, com liberdade de implementação (por exemplo, correr como multiplicador de velocidade aplicado à `velocity`, ou pular como um impulso vertical simples somado a `velocity.y`). Não há solução única.

Checklist

- ☐ Input Map do projeto com as Actions `move_forward`, `move_back`, `move_left`, `move_right`
- ☐ Orchestration `player.torch` lendo as quatro Actions e combinando-as em um vetor de direção
- ☐ `velocity` do Player atribuída antes da chamada a `move_and_slide`
- ☐ Player se move nas quatro direções e desliza corretamente ao encostar em obstáculos
- ☐ Scene testada com F6, sem erros
- ☐ Action adicional do desafio (correr, agachar ou pular) criada e funcional

Glossário

- **Input Map:** configuração global do projeto que associa teclas/botões físicos a Actions nomeadas.
- **Action:** nome lógico de uma intenção do jogador (ex.: `move_forward`), desacoplado da tecla física usada para acioná-la.
- **InputEvent:** evento gerado pelo Godot a cada interação do jogador com um dispositivo físico, traduzido automaticamente para a Action correspondente.
- **velocity:** propriedade do `CharacterBody3D` que define a direção e intensidade do movimento antes da chamada a `move_and_slide`.
- **PhysicsProcess:** evento chamado a cada frame de física, ponto correto para ler input e mover o `CharacterBody3D`.

Referências

- Godot Documentation — Inputs:
<https://docs.godotengine.org/en/stable/tutorials/inputs/index.html>

- Godot Documentation — Physics — CharacterBody3D:
https://docs.godotengine.org/en/stable/classes/class_characterbody3d.html
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Input System:
<https://docs.unity3d.com/Manual/com.unity.inputsystem.html>
- GDQuest: <https://www.gdquest.com/>